MTH251 2022S1 Exam

Feel free to contact me at maths_and_programming@outlook.sg if you need tuition. I provide university level tuition for the following SUSS units:

* ENG201 Linear Systems Analysis and Design
* ENG301 Microprocessor Programming
* ICT101 Computational Thinking
* ICT133 Structured Programming
* ICT162 Object Oriented Programming
* MTD215 Application of C++ in Multimedia
* ICT231 Data Structures and Algorithms I
* ICT232 Data Structures and Algorithms II
* MTH105 Fundamentals of Mathematics
* MTH107 Calculus I
* MTH108 Calculus II
* MTH316 Multivariable Calculus
* MTH319 Mathematical Methods I
* MTH320 Mathematical Methods II
* MTH207 Linear Algebra
* MTH219 Fundamentals of Statistics and Probability
* MTH220 Statistical Methods and Inference
* MTH355 Basic Mathematical Optimisation

Donaldson Tan

MTH251

Timed Online Assignment – January Semester 2022

Data Structures and Algorithms I

Saturday, 21 May 2022 **4:00 pm – 6:00 pm**

Time allowed: 2 hours

INSTRUCTIONS TO STUDENTS:

1. This Timed Online Assignment (TOA) contains **EIGHT (8)** questions and comprises **TWELVE (12)** pages (including cover page).

2. You must answer **ALL** questions.

3. If you have any queries about a question, or believe there is an error in the question, briefly explain your understanding and assumptions about that question before attempting it.

4. You MUST **submit your answers via Canvas (similar to TMA submission) at the end time of this TOA** (as stated on this cover page). The 15 minutes grace period as shown on Canvas is strictly meant for technical issues encountered during submission. Thereafter, you will not be able to submit your answers and you will be considered as having withdrawn from the course. **No appeal will be allowed.**

5. You are to include the following particulars in your submission: TOA Course Code, your Full Name, and SUSS PI Number. Name your submission file as TOACourseCode_FullName_StudentPI. Use underscore and NOT Space. *Example: TOAXYZ_RaphaelLee_T1923161 (omit D/O, S/O).*

6. Your submission should consist of only one file and **must not exceed 500MB** in size. The file must be a PDF or Microsoft Word file saved in .doc or .docx format. All answers are to be typed. Flowcharts and graphs may be scanned or photographed and embedded in the PDF or Word file provided it does not exceed the file size limit of 500MB.

7. For answers which cannot be typed and require to be hand-written, ensure that question number is clearly stated on each page. All uploaded hand-written answers must be clear, readable and complete. Marks will not be awarded for un-readable or incomplete images.

8. To prevent plagiarism and collusion, your submission will be reviewed by Turnitin. The Turnitin report will only be made available to the marker and you will not be able to view it.

9. The University takes plagiarism and collusion seriously, and your Turnitin report will be examined thoroughly as part of the marking process.

10. Marks will only be awarded if FULL working is shown.

Answer all questions. (Total 100 marks)

Question 1

(a) The number of operations executed by algorithm A and B are $\mathbf{12n^3 + 40n\log_2 n}$ and $\mathbf{5n^4 - 100n^2}$. Solve for n_0 such that A is better than B for $n \geq n_0$. Show the procedures.

(5 marks)

(b) Analyse the time complexity of the sorting algorithm shown below in the **best** and **worst** case:

```
def sort(alist):
    n = len(alist)
    for j in range(n - 1):
        count = 0
        for i in range(0, n - 1 - j):
            if alist[i] > alist[i + 1]:
                alist[i], alist[i + 1] = alist[i + 1], alist[i]
                count += 1
        if 0 == count:
            break
```

(5 marks)

Question 2

(a) What is the data type of 0xFF in Python?

(A) number
(B) hexint
(C) hex
(D) int

(2 marks)

(b) Which is NOT the core data type in Python?

(A) list
(B) tuple
(C) dictionary
(D) string
(E) class

(2 marks)

(c) Which of the following are true? (Select all the true statements.)

(A) All elements in a list must be of same type.
(B) A list may contain any type of object except another list.
(C) List is mutable.
(D) A given object may appear in a list more than once.
(E) There is no conceptual limit to the size of a list.
(F) These represent the same list:
['a', 'b', 'c']
['c', 'b', 'a']

(2 marks)

(d) Suppose you have the following tuple definition:

t = ('foo', 'bar', 'algo')

Which of the following statement replaces the 3rd element ('algo')?

(A) t[2] = 'x'
(B) t(2) = 'x'
(C) t(2) = None
(D) t[2:2] = ''
(E) None of the above

(2 marks)

(e) Which of the following are true of Python dictionaries? (Select all the true statements.)

(A) Dictionaries are mutable.
(B) A dictionary can contain any object type except another dictionary.
(C) All the keys in a dictionary must be of the same type.
(D) Dictionaries can be nested to any depth.
(E) Items are accessed by their position in a dictionary.
(F) Dictionaries are accessed by key.

(2 marks)

(f) Consider the following sequence of statement:

m = 123
n = m

Following the execution of above statements, state the number of objects and references created by Python.

(A) 2 objects, 1 reference
(B) 1 object, 2 references
(C) 2 objects, 2 reference
(D) 1 object, 1 reference

(2 marks)

(g) Consider the following nested list in Python:

x = [3.14, [1, 2, [1, ‘hello’, 2]], ‘world’]

What is the expression that returns the ‘e’ in ‘hello’?

(2 marks)

(h) Which type of copy is shown in the following Python code:

```
foo = [[10, 20], [30, 40], [50, 60]]
foo1 = list(foo)
```

(A) Shallow copy
(B) Deep copy
(C) memberwise
(D) None of the above

(2 marks)

(i) What is the runtime complexity of the list’s build-in .append() method?

(A) O(1)
(B) O(log n)
(C) O(n^2)
(D) O(n)

(2 marks)

Question 3

(a) Given a two-dimension array and a target, complete the code below to implement an algorithm in Python to find if the target is in the array:

```
def find(array, target):

    if not array:
        return

    row = len(array)
    col = len(array[0])

    for i in range(row):
        for j in range(col):
            # complete code here

    return False
```

(5 marks)

(b) Analyse the time complexity of the algorithm in Question 3(a).

(5 marks)

(c) If the elements in the array are sorted by row and by column, for example:

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \end{bmatrix}$$

Describe how to improve the find algorithm in Question 3(a) so that the time complexity can be optimized to O(n).

(5 marks)

Question 4

(a) Given that my_s is an empty instance of the Stack ADT, the following operations are performed. Interpret the operations and write down the elements in my_s.

```
my_s.push(3)
my_s.push(7)
my_s.pop()
my_s.push(11)
my_s.top()
my_s.push(5)
my_s.push(2)
my_s.size()
my_s.pop()
my_s.pop()
```

(5 marks)

(b) Given that the my_s in Question 4(a) is implemented using an array-based list in Python.

(i) After the execution of the operation in Question 4(a), the first element of my_s has the memory address of 1024. What is the memory address of the last element in my_s if each memory address occupies four bytes? Justify your answer.

(5 marks)

(ii) What is the size of the underlying dynamic array if one more push operation is performed? Justify your answer.

(5 marks)

Question 5

Deque is an enhanced version of Queue ADT that allows insert and delete at both ends:

```
class Deque():

    class Node():
        def __init__(self, data, prev, next):
            self.data = data
            self.prev = prev
            self.next = next

    def __init__(self):
        self.front = self.rear = None
        self.size = 0

    def is_empty(self):
        return self.front is None

    def insert_front(self, data):
        n = self.Node(data, None, None)
        if self.front is None:
            self.front = self.rear = n
        else:
            node.next = self.front
            self.front.prev = n
            self.front = n
        self.size = self.size + 1

    def insert_rear(self, data):
        n = self.Node(data, None, None)
        if self.rear is None:
            self.front = self.rear = n
        else:
            n.prev = self.rear
            self.rear.next = n
            self.rear = n
        self.size = self.size + 1

    def delete_front(self):
        if self.is_empty():
            return
        temp = self.front
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        else:
            self.front.prev = None
        self.size = self.size - 1
        return temp
```

```
        def delete_rear(self):
        if self.is_empty():
            return
        temp = self.rear
        self.rear = self.rear.prev
        if self.rear is None:
            self.front = None
        else:
            self.rear.next = None
        self.size = self.size - 1
        return temp

    def get_front(self):
        if self.is_empty():
            return
        return self.front.data

    def get_rear(self):
        if self.is_empty():
            return
        return self.rear.data

    def earse(self):
        self.rear = None
        while self.front:
            temp = self.front
            self.front = temp.next
        self.size = 0
```

(a) Analyse the time complexity of insert_front, insert_rear, delete_front and delete_rear functions. Explain your analysis.

(5 marks)

(b) In storing the web browser history, the user needs to remove some recently visited URLs from the history while system needs to delete old URLs from this history before it grows too big. Considering between Deque and Queue, which data structure would you choose? Discuss your reason.

(5 marks)

Question 6

Consider the following Python implementation of a new Stack class which provides a new method (get_min) to return the minimum element in the stack:

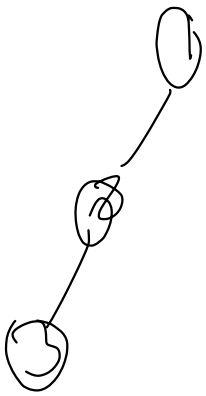
```
class NewStack():

    def __init__(self):
        self.data = []
        self.min = []

    def push(self, n):
        self.data.append(n)
        if n <= self.get_min():
            self.min.append(n)

    def get_min(self):
        if len(self.min) == 0:
            return None
        return self.min[-1]

    def pop(self):
        value = self.data.pop()
        return value
```

Identify the bugs in the code and correct the same.

(10 marks)

Question 7

Based on the tree shown in Figure Q7, answer the following questions with explanation:



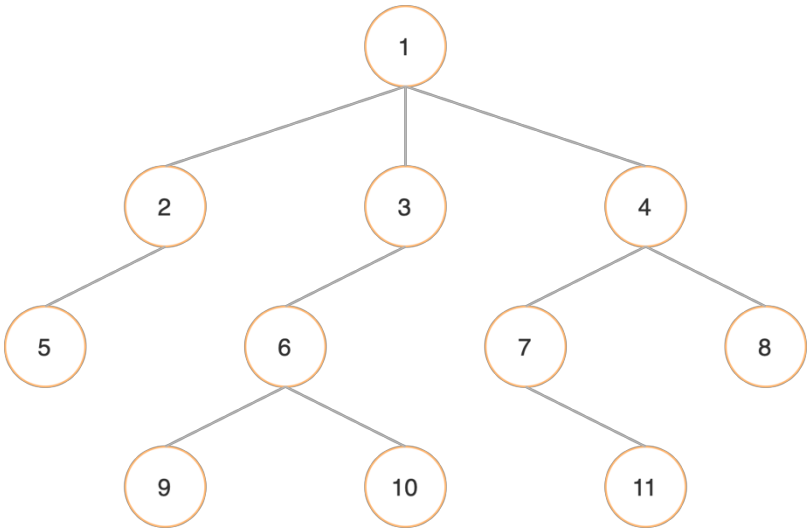


Figure Q7

(a) Is it a binary tree?

(2 marks)

(b) How many internal nodes in the tree?

(2 marks)

(c) What is the degree of node 6?

(2 marks)

(d) What is the height of node 4?

(2 marks)

(e) What are the siblings of node 7?

(2 marks)

(f) How many variants are there for depth-first traversal?

(2 marks)

Question 8

Consider the following Python implementation of a tree node:

```
# A node contains the value, left and right pointers
class TreeNode:
    def __init__(self,data):
        self.data = data
        self.left = self.right = None
```

(a) Complete the code below to solve a binary tree inversion by using recursive algorithm:

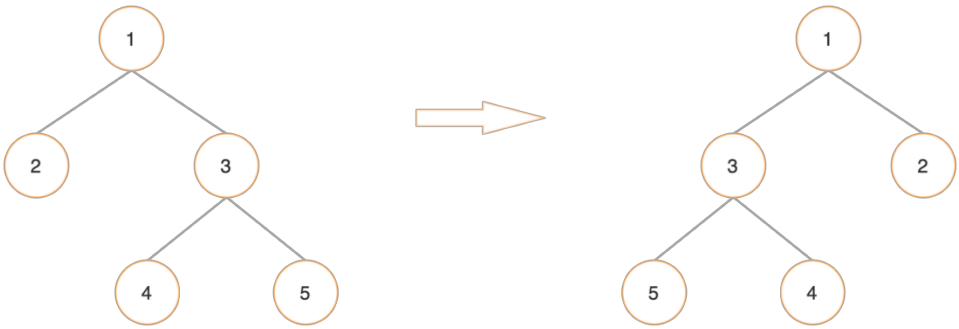


Figure Q8(a)

```
def invert(node):

    if (node == None):
        return
    else:
        # complete code here

root = TreeNode(1)
root.left = TreeNode(2)
root.right = TreeNode(3)
root.right.left = TreeNode(4)
root.right.right = TreeNode(5)
# convert tree to its mirror
invert(root)
```

(5 marks)

(b) Analyse the time complexity of the algorithm in Question 8(a).

(5 marks)

----- END OF PAPER -----

1 (a) $T_A = 12n^3 + 40n \log_2 n$

$T_B = 5n^4 - 100n^2$

A better than B

$\Leftrightarrow T_A \leq T_B$

$12n^3 + 40n \log_2 n \leq 5n^4 - 100n^2$

$40n \log_2 n \leq 5n^4 - 12n^3 - 100n^2$

since $n > 0$

$40 \log_2 n \leq 5n^3 - 12n^2 - 100n$

let $P(n) = 40 \log_2 n \leq 5n^3 - 12n^2 - 100n$

n	$40\log_2 n$	$5n^3-12n^2-100n$	$P(n)$
1	0	-107	F
2	40	-208	F
3	63.4	-273	F
4	80	-272	F
5	92.9	-175	F
6	103.4	48	F
7	112.3	427	T
8	120	992	T
9	126	173	T

$n \geq 7 \quad \therefore n_0 = 7$

(b) best case is when sorting a sorted list of ascending order whereby the external for-loop only run once.

best case time complexity = 1 assignment + 1 comparison + 1 assignment + $(n-1)$ comparison + $(n-2)$ comparison + 1 comparison + 1 break

$= 2n+2$ steps

$= O(n)$

worst case is when sorting a sorted list of descending order

worst-case time complexity = 1 assignment + n comparisons + $\underbrace{(n-1)}_{\text{external loops}}$ (1 assignment + 1 comparison) + 1 break +

$\underbrace{[1+2+\ldots+(n-1)]}_{\text{internal loops}}$ (1 comparison + 3 assignments + 1 addition) +

$[2+3+\ldots+n]$ comparisons

$= n+2 + 2(n-1) + \frac{n(n-1)}{2}(5) + \left[\frac{n(n+1)}{2} - 1\right]$

$= O(n^2)$

2(a) D int

(b) E class

(c) C, D, E

(d) E

(e) A, D, E

(f) B

(g) x[1][2][1][1]

(h) shallow copy

(i) A

3(a)
```
for i in range(row):
    for j in range(col):
        if array[i][j] == target
            return True
```

(b) Given an array with m rows and n columns,
let $N = \max(m,n)$

worst case happens when target not found

worst-case time complexity

= 1 comparison + 1 assignment + 1 array read + 1 assignment + (m+1) comparisons + (n+1) comparisons + m.n comparisons + 1 return

$= mn + m + n + 7$

$= O(mn)$

$= O(N^2)$

(c) Given an array of m rows and n columns, the range of each row is given by:
upper bound of each row = first element of each row
lower bound of each row = last element of each row

A linear search is then employed to identify the likely row where the target is located by comparing it using lower bound $\leq$ target $\leq$ upper bound. The time complexity for this step is $O(m)$. If no likely row is found, return False.

After the likely row is identified, then a linear search is performed on the likely row to find the target. If the target is found, return True, else return False. The time complexity for this step is $O(n)$.

The overall time complexity is $O(m) + O(n) = O(N) + O(N) = O(N)$ where $N = \max(m,n)$

4(a)

Operation	Response	Contents
my_s.push(3)	True	3 | | | | |
my_s.push(7)	True	3 | 7 | | | |
my_s.pop()	7	3 | | | | |
my_s.push(11)	True	3 | 11 | | | |
my_s.top()	11	3 | 11 | | | |
my_s.push(5)	True	3 | 11 | 5 | |
my_s.push(2)	True	3 | 11 | 5 | 2 |
my_s.size()	4	3 | 11 | 5 | 2 |
my_s.pop()	2	3 | 11 | 5 | |
my_s.pop()	5	3 | 11 | | |

(b) i. memory address $= 1024 + 4$
$= 1028$

ii. Assuming the dynamic array was initialized at size 5, one more push operation would result in the consumption of a 3rd 4-byte memory block. This doesn't trigger the resize operation, so the size of the underlying dynamic array remained at $5 \times 4 = 20$ bytes.

5(a) time complexity of node creation = 3 assignments = 3

time complexity of insert_front() = 3 assignments (1 x node creation) + 1 assignment + 1 comparison + 3 assignments + 1 addition + 1 assignment

= 10 steps

= $O(1)$

time complexity of insert_rear() = 3 assignments (1 x node creation) + 1 assignment + 1 comparison + 3 assignments + 1 addition + 1 assignment

= 10 steps

= $O(1)$

time complexity of delete_front() = 1 comparison + 2 assignments + 1 comparison + 1 assignment + 1 subtraction + 1 assignment + 1 return

= 8

= $O(1)$

time complexity of delete_rear() = 1 comparison + 2 assignments + 1 comparison + 1 assignment + 1 subtraction + 1 assignment + 1 return

= 8

= $O(1)$

(b) Choose Deque. URLs could be added in chronological order using the insert_rear() operation, while older URLs could be removed using the delete_front() operation.

6. This is a stack that keeps track of the minimum element.
None is assumed to be an invalid element since it cannot be compared against other element

The bug is at line 9. It is caused by the fact get_min() returns either an element or None and the code at line 9 cannot handle None.

<u>Fix is</u>

```
def push(self,n):
    self.data.append(n)
    m = self.get_min()
    if m is None or n <= m:
        self.min.append(n)
```

Another bug is in the pop() function as it doesn't update self.min.
and check if self.data is empty.

<u>Fix is</u>

```
def pop(self):
    if len(self.data) == 0:
        return None
    value = self.data.pop()
    if value == self.min[-1]:
        self.min.pop()
    return value
```

7(a) It is not a binary tree because the root has 3 children.

(b) 6 internal nodes because only nodes 1, 2, 3, 4, 6, 7 have children.

(c) degree of node 6 = 2 because it has 2 children

(d) height of node 4 = 2

(e) sibling of node 7 = node 8 since they share same parent

(f) pre-order, in-order and post-order

5(a)

```
def invert_node(node):
    if node == None:
        return
    else:
        node.left, node.right = node.right, node.left
        invert_node(node.left)
        invert_node(node.right)
```

(b) During each node visit, the left and right pointers are swapped.

Each node visit is $O(1)$ time complexity while every node is visited once.

$\therefore$ time complexity $= n \times O(1) = O(n)$ where n is number of nodes